

Archiso Workbench

A replacement for `automated_zfs_iso_script.sh`, with a personal reinstall payload. This builds **live Arch installation/rescue media**. The ISO filesystem itself is not ZFS, and this is not an unattended root-on-ZFS installer.

Two presets are included:

Preset	Installed software beyond upstream releng
rescue	Editors, storage tools, ddrescue, TestDisk, hardware diagnostics
personal	Editors, Git/GitHub CLI, Zsh, Starship, tmux, Python, Ruff, ShellCheck, shfmt, compilers

Both carry your payload and helper scripts. Add `--zfs` to either preset for ZFS support. A rescue image without ZFS can be built without any AUR packages. The personal preset is a terminal environment; it does not start Hyprland or SDDM.

1. Build host

Use an updated **x86_64 Arch Linux machine or dedicated Arch VM**, with working DNS/mirrors and enough disk space. Start with roughly 25–40 GiB free; a larger payload or DKMS build may need more. The builder does not install host dependencies.

```
sudo pacman -Syu --needed archiso arch-install-scripts python \
    base-devel git xorriso
```

Extract this package somewhere writable by your normal user. Do not run the wrapper with sudo. It requests sudo for the privileged build and inspection steps.

```
cd archiso-workbench
./build-iso.sh --help
./build-iso.sh --preset personal          # Stage only: no ISO yet
./build-iso.sh --preset personal --build  # Fresh stage and actual build
```

Staging requires the installed `releng` profile, but no network or sudo when local packages are not used. Every invocation creates a new run. Staging and building are separate fresh runs: edit the source payload/package lists before `--build`; edits to an earlier staged profile will not be reused.

The successful run contains:

- `out/*.iso` and `out/SHA256SUMS`;
- `manifest.json`: requested packages and hashes of profile files/local archives;
- `installed-packages.txt`: actual package versions in the image;
- `build.log`, `boot-report.txt`, and optionally `zfs-check.log`;
- the generated profile and retained work directory.

A failed verification leaves the image under `candidate/`, not `out/`. Do not use a candidate as a validated release. No broad cleanup or host `pacman.conf` changes are performed. The

build still runs trusted packages/hooks with privileges: use a VM if you want stronger isolation from your ordinary host.

2. ZFS packages

ZFS is supplied outside Arch's official repositories. This version deliberately accepts **reviewed local `zfs-dkms` and `zfs-utils` package archives**, instead of silently trusting a third-party repository or building remote scripts unattended. It adds the ISO's `linux-headers`, DKMS, and build tools to the image.

Obtain the current AUR recipes for `zfs-utils` and `zfs-dkms`, inspect their PKGBUILDs, install files, patches, source URLs and signature checks, and build them as a normal user. For example, in a dedicated Arch build VM:

```
git clone https://aur.archlinux.org/zfs-utils.git
git clone https://aur.archlinux.org/zfs-dkms.git
less zfs-utils/PKGBUILD
less zfs-dkms/PKGBUILD
```

After reviewing all referenced files, use `makepkg -s` in each checkout. Resolve any AUR-only dependencies deliberately. If the second recipe requires the built utilities, install that reviewed archive **in the build VM**, then build again. The exact dependency graph is defined by those current recipes, not this guide. Do not use `--skipgpcheck`. Prefer a clean chroot for maintained package builds; see Arch's developer tooling. Keep the recipe commit IDs with your build notes.

Copy only the intended archives into a dedicated directory:

```
mkdir -p local-packages
# Copy your reviewed zfs-utils and zfs-dkms *.pkg.tar.zst archives here.
./build-iso.sh --preset rescue --zfs \
    --local-packages ./local-packages --trust-local-packages --build
./build-iso.sh --preset personal --zfs \
    --local-packages ./local-packages --trust-local-packages --build
```

Every package archive in that directory is installed, so do not point this option at your entire pacman cache. Only `.pkg.tar.zst` is supported. Multiple versions of the same package are rejected. Include required local dependencies too.

`--trust-local-packages` explicitly allows unsigned archives in the run's local repository. It does **not** disable official package signatures or trust unknown signers. Signed local archives need their signing key already trusted by the build host's pacman keyring. No keys are fetched/imported automatically. The live `pacman.conf` has no broken reference to this host-only repository.

Kernel compatibility

DKMS is not a guarantee that a new Linux kernel is supported by a given ZFS release. After building, the wrapper checks the ISO's installed `linux` kernel, matching headers, ZFS module vermagic, and module dependency resolution. It never uses the build host's `uname -r` as the target kernel. A DKMS failure can survive pacman's post-transaction

phase; the additional check prevents publication of that candidate. It does not prove that the module will load or operate correctly.

If the current kernel is unsupported, wait for compatible packages or choose a **known compatible complete Arch Archive snapshot**:

```
./build-iso.sh --preset rescue --zfs \  
  --local-packages ./local-packages --trust-local-packages \  
  --snapshot YYYY/MM/DD --build
```

Replace YYYY/MM/DD with a researched date. Build local packages against that same snapshot in a separate matching environment. The snapshot applies to all official repositories in the profile and live system. A date inferred from an HTML directory listing is not a compatibility check. Current archiso itself may also be incompatible with sufficiently old snapshots. This tool does not claim bit-for-bit reproducible builds or support arbitrary historical dates.

This release retains the standard `linux` kernel and upstream boot references. It does not implement a kernel switch to `linux-lts`; that requires corresponding profile and boot configuration work. A live image with ZFS tools does not require root-on-ZFS initramfs hooks. Installed-system ZFS boot configuration is a separate step; consult the OpenZFS root-on-ZFS guide.

3. Put your usual files in the ISO

Source directory	Location/meaning inside the ISO
<code>overlay/common/</code>	Files applied to the live filesystem root
<code>overlay/zfs/</code>	Additional live files when <code>--zfs</code> is selected
<code>overlay/custom/</code>	Optional additions for your own live setup
<code>payload/home/</code>	<code>/opt/reinstall-kit/payload/home/</code> : future user-home files
<code>payload/system/</code>	<code>/opt/reinstall-kit/payload/system/</code> : future system files
<code>packages/extra.txt</code>	Additional official or supplied local package names

The bundled templates use Danish keyboard layout, `en_DK.UTF-8`, a Zsh fragment, a Hyprland keyboard fragment, a two-space/81-column EditorConfig, and a `cpb` backup-copy helper. These are templates, not a recovered copy of your dotfiles. The live locale is generated through the distribution's `glibc` installation hook.

For example, capture your existing Neovim settings and selected scripts:

```
./overlay/common/usr/local/bin/iso-config capture \  
  --payload ./payload \  
  --path .config/nvim \  
  --path .local/bin/my-install-helper
```

Run `capture` without `sudo`. It refuses overwriting existing template files; review and remove the particular template first if you want to replace it. Hidden files are included

within the paths you explicitly select. Symlinks and special files are rejected rather than copied from unexpected locations. If a configuration is symlink-managed, explicitly copy the reviewed real file into `payload/home`.

You can place additional scripts directly in `overlay/custom/usr/local/bin` and mark them executable. Their interpreters and dependencies must appear in a package list. Scripts in `payload/home/.local/bin` are carried as reinstall files; they are not automatically available on the live shell's PATH.

Do not include SSH/GPG keys, LUKS key files, Wi-Fi passwords, API tokens, browser profiles, Codex authentication files, or whole home-directory snapshots. The scanner catches some obvious secret files but cannot certify that a file is safe. An ISO is readable by anyone who possesses it; file permissions do not encrypt it. No credentials are required to boot this media. Inherited releng services/login settings are retained; inspect the generated profile before distributing it.

4. Reinstall and apply configurations

Boot the ISO, run `iso-welcome`, and inspect the machine with `iso-doctor`. Partition/mount/install Arch yourself, with all intended target filesystems mounted below `/mnt`. Create the target user and home before applying home files. Packages installed in the **live image** are not automatically installed into the new system, and package archives are not retained as an offline install cache. Use `pacstrap`/the target package manager separately, including compatible ZFS packages if the target needs them.

From the live ISO after installing Arch and creating user heini:

```
iso-config apply --target /mnt --user heini --overwrite
```

Read the preview, then copy the home files:

```
iso-config apply --target /mnt --user heini --overwrite --apply
```

Separately preview/apply the small system template set:

```
iso-config apply --target /mnt --scope system --overwrite
```

```
iso-config apply --target /mnt --scope system --overwrite --apply
```

The target must be a mount point containing an Arch installation. `/` is refused. The helper reads the target `passwd` file for the user's actual UID/GID/home; it does not assume UID 1000. Existing files require `--overwrite` and receive backups under `/mnt/var/backups/iso-kit/apply-*/files/`. The adjacent manifest lists target-relative paths and distinguishes newly created files from replacements. Writes are atomic per file, not transactional across the whole set. Interrupted operations retain backups and the journal.

For restoration, inspect the manifest, copy the specific old files from `files/` back to their corresponding target-relative paths with `cp -a`, and remove newly created files only after checking they have not subsequently been edited. Backups must remain available until you accept the result. This helper does not restore ACLs or provide an automatic rollback transaction.

Generate `en_DK.UTF-8` in the **target** `/etc/locale.gen` and run `arch-chroot /mnt locale-gen`. Set the target timezone separately. Source the Zsh fragment from `.zshrc` and the Hyprland fragment from your existing configuration if wanted. Boot files, `fstab`, `crypttab`,

pool layout, GPU settings and host-specific identifiers are deliberately not guessed from an old disk layout.

5. VM verification

Install `qemu-desktop` and `edk2-ovmf` on your test host if needed. Identify the matching non-Secure-Boot OVMF firmware pair installed by your version:

```
pacman -Ql edk2-ovmf
./bin/test-iso.sh /absolute/path/image.iso \
    /absolute/path/OVMF_CODE.fd /absolute/path/OVMF_VARS.fd
```

The launcher attaches only the ISO, with explicit CD boot priority and private UEFI variables. It uses KVM if accessible, otherwise slower TCG. It does not need or attach a physical disk. Do not mix different OVMF code/variable sizes.

Inside the VM:

```
iso-doctor
locale -a
command -v nvim git python iso-config
ls /opt/reinstall-kit/payload/home
# For the ZFS variant only:
modprobe zfs
zfs version
zpool import          # With no pool argument: discovery only
```

Require UEFI boot, the intended locale, your payload, and successful ZFS loading before trusting the build. Use a separate VM with disposable virtual disks to test pool creation/import and an actual target installation. Test BIOS separately if you need it. Neither an El Torito report nor module `vermagic` proves bootability. Secure Boot key enrollment/signing is not configured by this project.

ZFS cache/scan import services are masked in the live image. No script imports pools automatically. Do not add automatic force-import behavior to rescue media.

6. Optional USB writing

First check the checksum from inside the ISO output directory:

```
sha256sum -c SHA256SUMS
ls -l /dev/disk/by-id/usb-*
python3 ./bin/write-usb.py /absolute/path/image.iso \
    /dev/disk/by-id/usb-YOUR_DEVICE
```

That is only a preview. Select the **whole disk**, not a `-partN` path. When ready:

```
sudo python3 ./bin/write-usb.py /absolute/path/image.iso \
    /dev/disk/by-id/usb-YOUR_DEVICE --write
```

Writing destroys the contents of that USB disk. The helper requires USB transport, a serial, adequate size, no mounted descendants or active swap, and a typed serial confirmation. It then compares an ISO-length read-back SHA256. It never unmounts anything.

These checks cannot know whether an unmounted disk contains valuable backups: verify its identity yourself. Do not unplug/reconnect devices mid-write.

7. Verification scope and changes from the original

See `docs/VALIDATION.md` for checks actually performed on this deliverable. The full Arch build, ZFS compilation, firmware boot, and physical USB write must be validated on your Arch machine/VM. No ready-made boot-tested ISO is included.

The rewrite removes host pacman replacement, HTML-date scraping, AUR helper installation, skipped PGP checks, obsolete community-repository manipulation, implicit broad deletion, and unguarded disk writing. It adds help, fresh runs, strict error handling, explicit trust boundaries, package manifests, configurable payloads, backed-up application, and separate VM/USB tools.

Sources

Checked against upstream documentation on 7 September 2026:

- Archiso profile format
- Archiso build implementation
- OpenZFS on Arch
- Building OpenZFS
- Arch clean-chroot package builds

The installed Archiso profile is the authority for boot files. Upstream APIs, package availability, and ZFS/kernel compatibility can change between builds.